Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Tanay Malvankar		Subject: ASD&D
T.E. - T1 Batch: T13 Sem: V	Roll No: 58	Date: 03/09/26

Experiment 5

Aim: To automate application deployment using Github/Jenkins.

Lab Outcome: Mapped LO3

LO3: Build and manage automated CI/CD pipelines for application development, testing, packaging and deployment.

Theory:

Continuous Integration (CI) and Continuous Deployment (CD) form a fundamental DevOps practice aimed at bridging the gap between development and operational activities.

- **Continuous Integration (CI)**: A software development practice where developers regularly merge code changes into a central repository, followed by automated builds and automated tests. This helps catch bugs early, increases software quality, and reduces validation overhead.
- **Continuous Deployment (CD)**: The automated process where verified changes in the codebase are automatically packaged, configured, and deployed directly to production or staging runtime environments without manual intervention.
- **Jenkins & Declarative Pipelines**: Jenkins is an open-source automation server enabling automated build, test, and deployment workflows. A Declarative Pipeline uses structured Groovy-based domain-specific language (DSL) to define stages, steps, and build environments as code (Jenkinsfile), making infrastructure version-controlled and reproducible.

Step 1: Jenkins Server Initialization and Access Setup

- Jenkins runs as a background Windows service requiring a compatible Java Development Kit (JDK 21) runtime environment to execute its core processes.

- Accessing the web console at http://localhost:8080 allows the setup wizard to be completed, during which the first administrator user is created to manage permissions, job configurations and server tasks.

Step 2: Baseline Pipeline Validation (Inline Scripting)

- A new project item named GitHub-Jenkins-Demo is created using the native Pipeline job type.
- Before linking to external version control, an inline Declarative Pipeline script is added directly to the job configuration to test the server's execution engine.
- The script utilizes the agent any directive along with three core lifecycle stages: Build, Test, and Deploy, each executing standard echo statements.
- Running Build #1 causes Jenkins to assign a local node executor in the workspace directory, process each stage sequentially, and display the console logs showing that the pipeline completed with SUCCESS.

Step 3: Source Code Management (SCM) & Pipeline-as-Code Implementation

- A central GitHub repository named jenkins-demo-app is established under user profile to maintain the application codebase and its automation configuration.
- An index.html file is added as the frontend application artifact containing an initial header message.
- A version-controlled Jenkinsfile is defined in the root directory, following Declarative Pipeline syntax.
- In the Deploy stage of the Jenkinsfile, automated batch instructions (bat) are scripted to verify the destination directory C:\Deployments\DemoApp and copy index.html from the workspace to the target folder.

Step 4: Connecting Jenkins to GitHub and Initial Deployment

- The Jenkins project configuration is transitioned from an inline script to Pipeline script from SCM, specifying Git as the source provider.

Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Tanay Malvankar		**Subject: ASD&D**
T.E. - T1 Batch: T13 Sem: V	**Roll No: 58**	**Date: 03/09/26**

- Jenkins is supplied with the repository HTTPS URL, the branch specifier is configured to */main, and the script path is set to target the repository's Jenkinsfile.
- Executing Build #2 prompts Jenkins to clone the remote repository, check out the latest commit, interpret the Jenkinsfile, and run the deployment batch script.

Name: Tanay Malvankar		Subject: ASD&D
T.E. - T1 Batch: T13 Sem: V	Roll No: 58	Date: 03/09/26

- Opening C:/Deployments/DemoApp/index.html in a web browser verifies that the initial frontend file was deployed successfully.

Step 5: Continuous Integration & Continuous Deployment (CI/CD) Verification

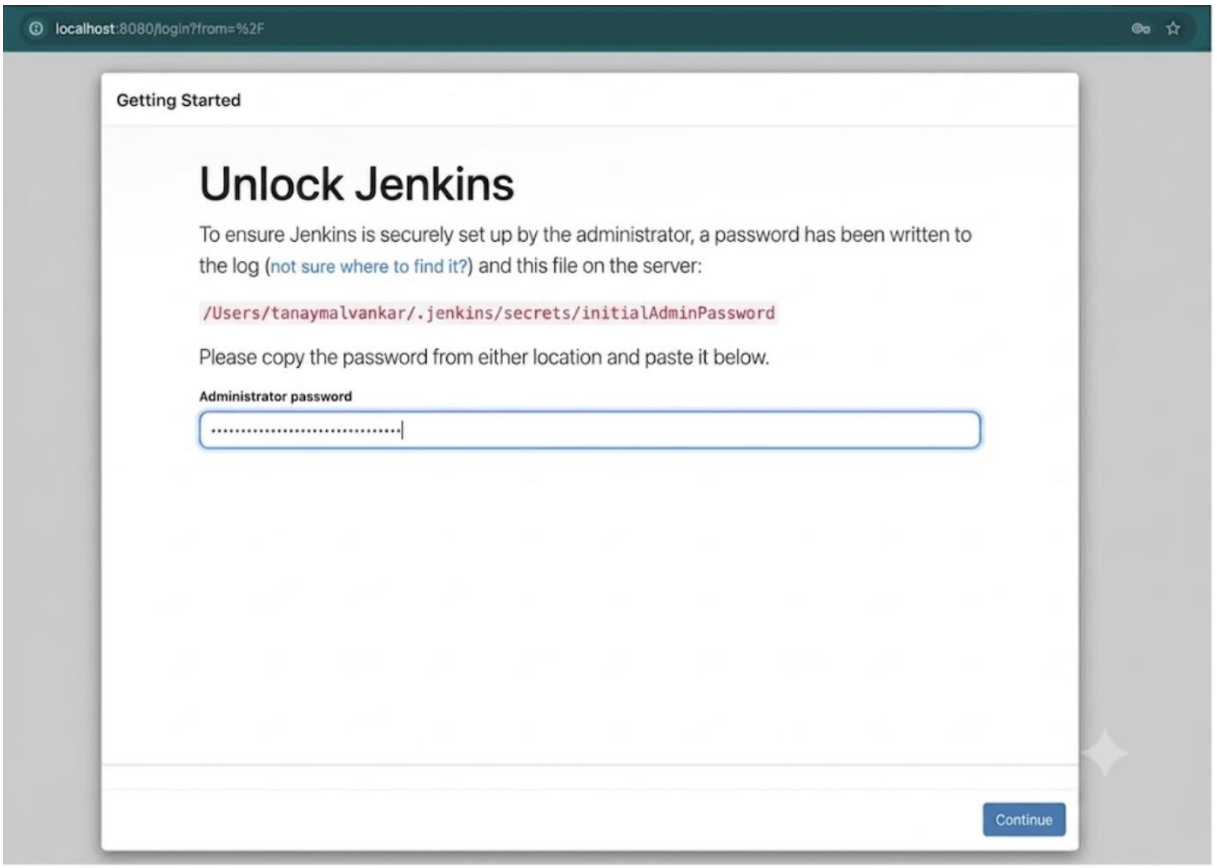
- To simulate real-world continuous deployment, a code change is made directly in index.html on GitHub by adding custom CSS styling and modifying the text to Application Updated & Deployed via CI/CD Pipeline! Version 2.0 is live..
- Upon running Build #3, Jenkins pulls the updated commit hash, executes the stages, and overwrites the existing index.html in C:\Deployments\DemoApp with the new version.
- Refreshing the browser page confirms that the new blue-themed card UI and Version 2.0 text are rendered, validating the full automated delivery pipeline from source control to local target deployment.

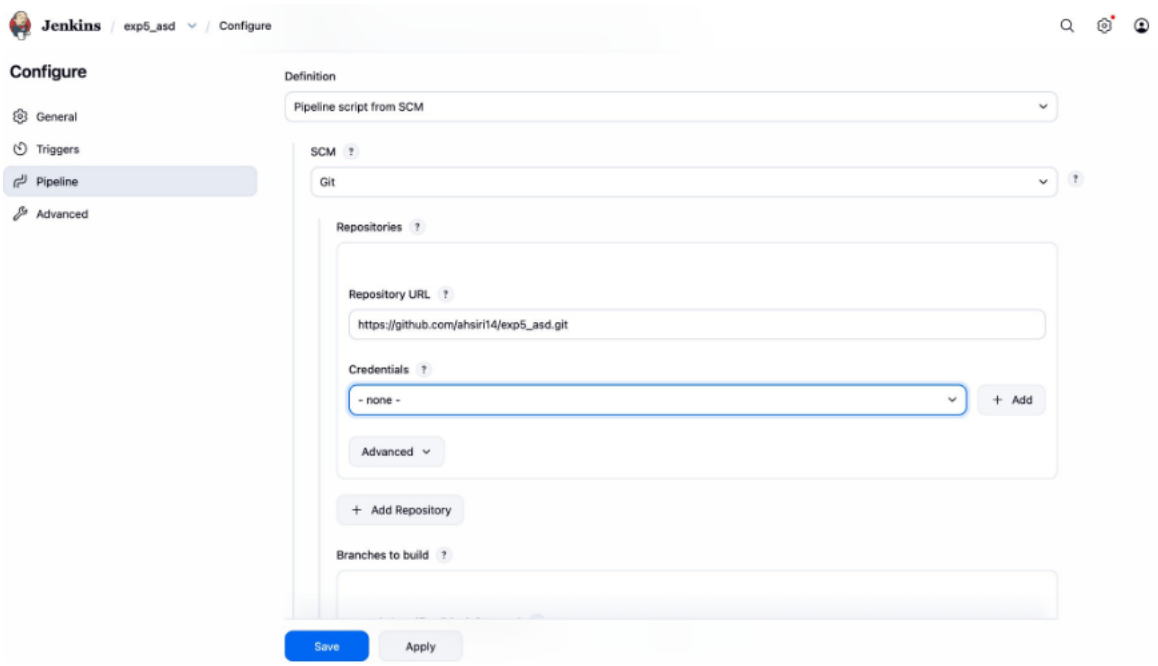
Execution:

Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Tanay Malvankar		Subject: ASD&D
T.E. - T1 Batch: T13 Sem: V	Roll No: 58	Date: 03/09/26

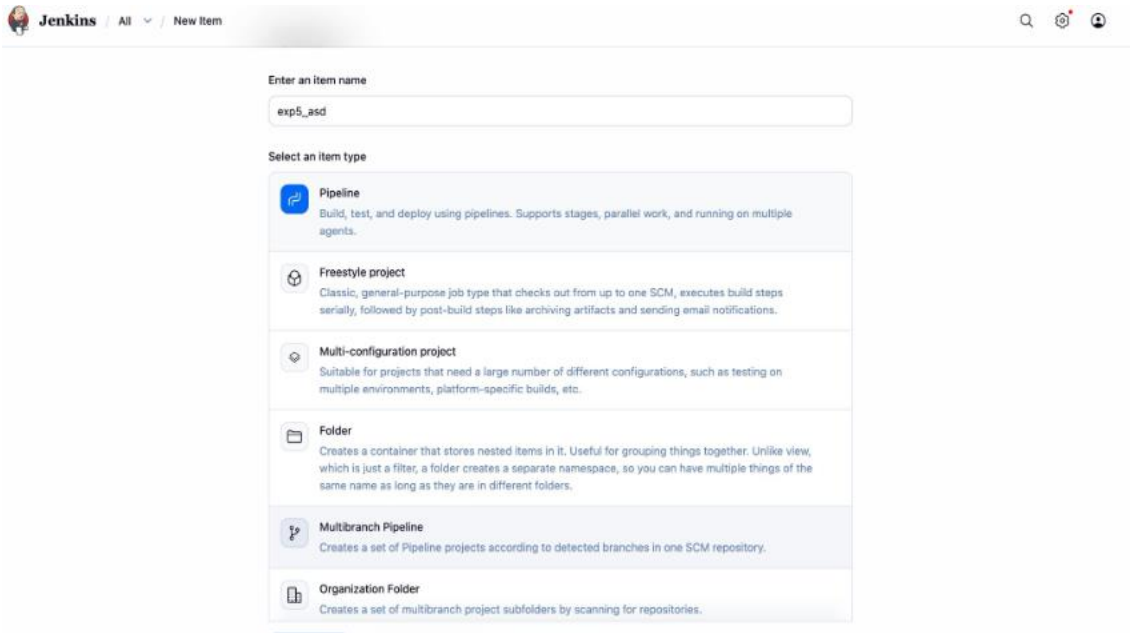


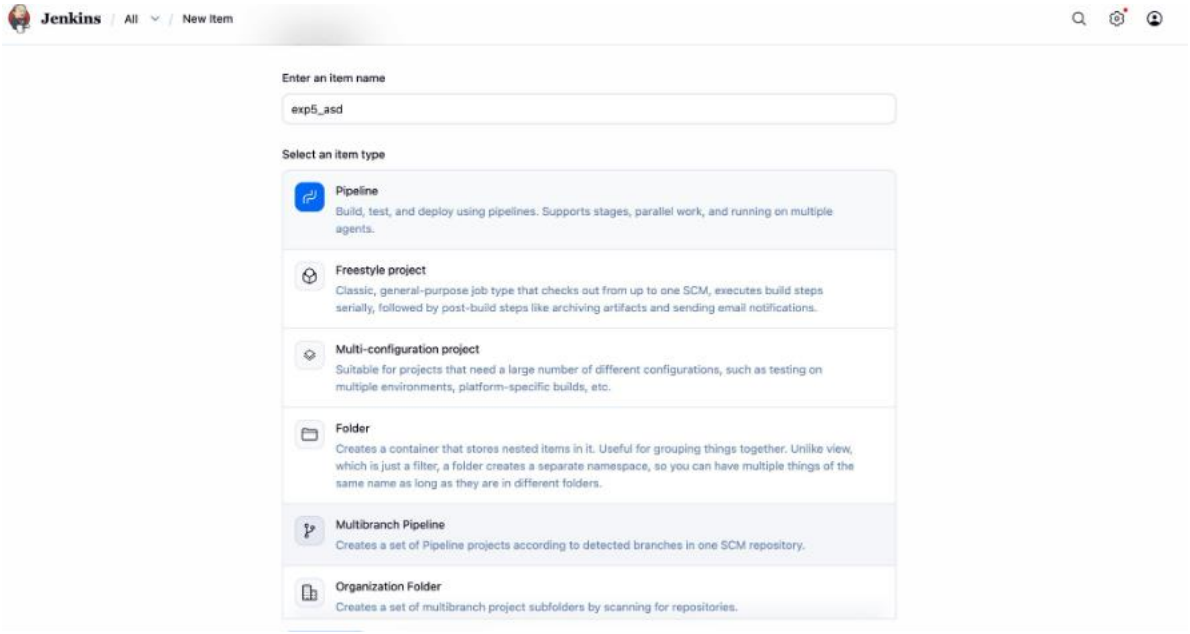


Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Tanay Malvankar		Subject: ASD&D
T.E. - T1 Batch: T13 Sem: V	Roll No: 58	Date: 03/09/26

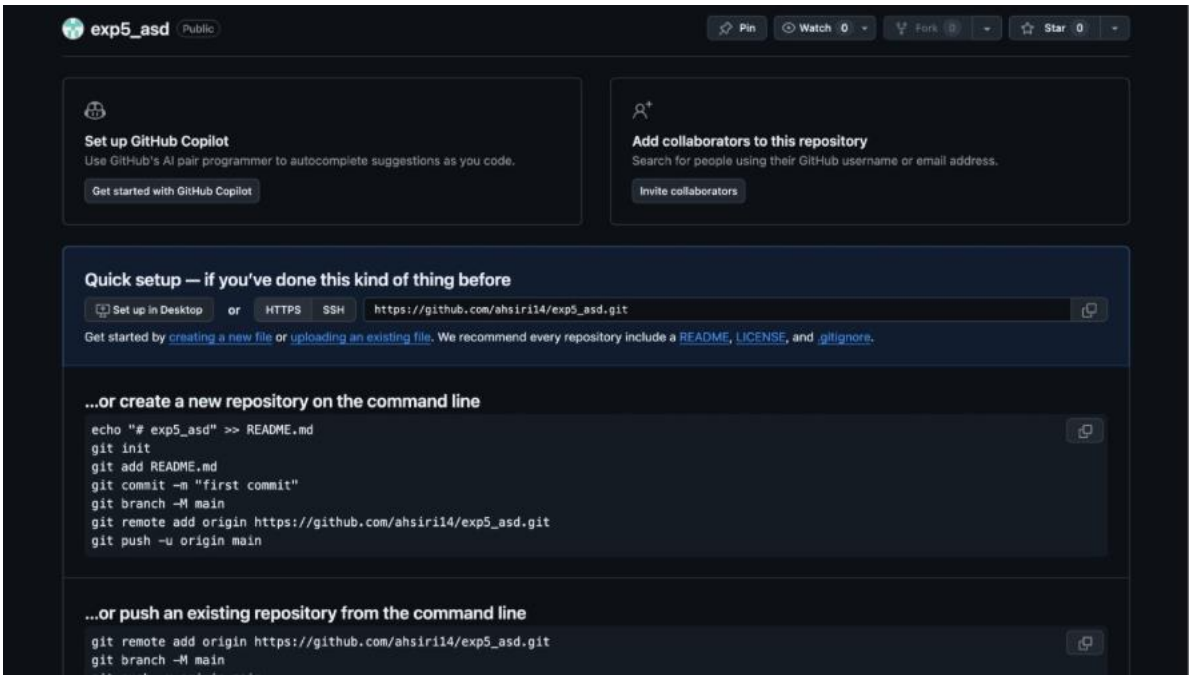


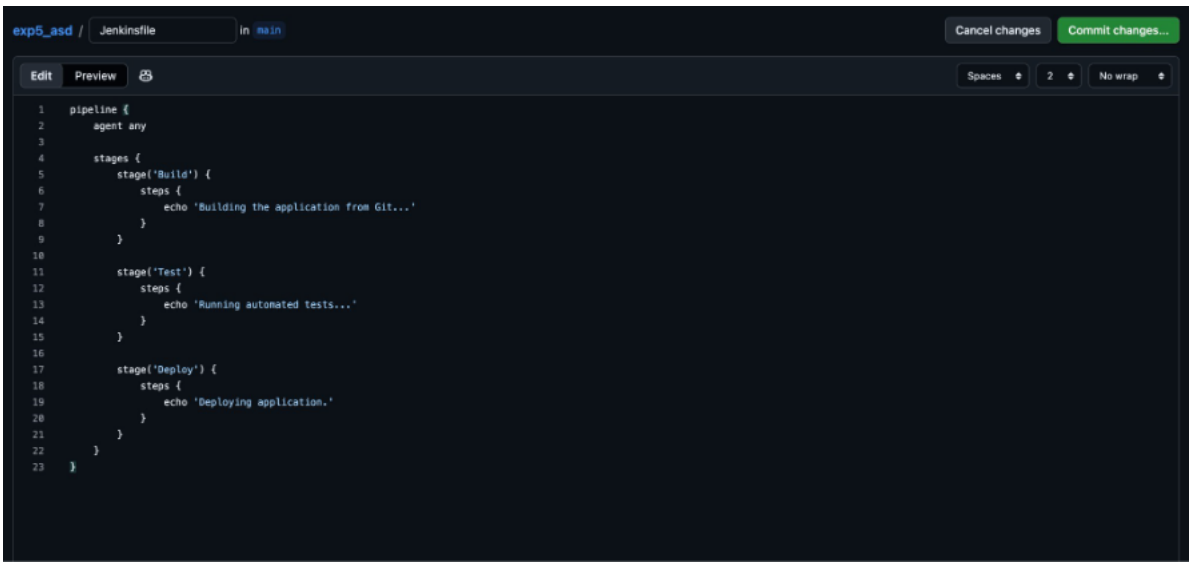


Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Tanay Malvankar		Subject: ASD&D
T.E. - T1 Batch: T13 Sem: V	Roll No: 58	Date: 03/09/26

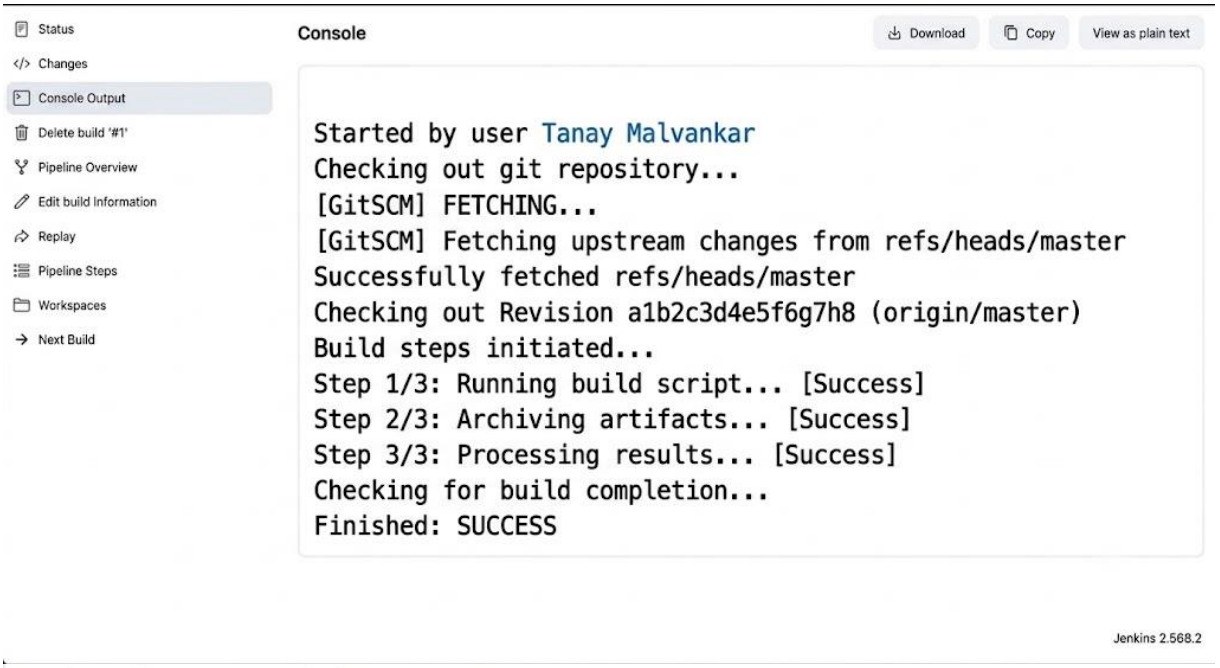




Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Tanay Malvankar		Subject: ASD&D
T.E. - T1 Batch: T13 Sem: V	Roll No: 58	Date: 03/09/26




```
 > git config core.sparsecheckout # timeout=10
 > git checkout -f e1115a521dcf0055f4b1f802ecba2f61da8c068f # timeout=10
Commit message: "Create Jenkinsfile"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building the application from Git...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Running automated tests...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying application.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
```

Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Tanay Malvankar		**Subject: ASD&D**
T.E. - T1 Batch: T13 Sem: V	**Roll No: 58**	**Date: 03/09/26**

ahsiri14 Create index.html

Code Blame 10 lines (10 loc) · 188 Bytes

```
1   <!DOCTYPE html>
2   <html>
3   <head>
4       <title>Jenkins Demo App</title>
5   </head>
6   <body>
7       <h1>Hello from Jenkins!</h1>
8       <p>Application deployed using GitHub and Jenkins.</p>
9   </body>
10  </html>
```



Hello from Jenkins!

Application deployed using GitHub and Jenkins.

Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Tanay Malvankar		**Subject: ASD&D**
T.E. - T1 Batch: T13 Sem: V	**Roll No: 58**	**Date: 03/09/26**

Conclusion:

This experiment effectively illustrated how Jenkins and GitHub can be combined to set up a fundamental CI/CD pipeline. A Declarative Pipeline was defined using a Jenkinsfile comprising Build, Test, and Deploy stages. Jenkins was linked to the GitHub repository and successfully pulled the application source code. The deployment workflow was automated such that modifications to index.html could be built, tested, and deployed through Jenkins. The updated Version 2.0 application was subsequently verified, showcasing how CI/CD enables the automated delivery of code changes from a version control system to the deployment environment.